



Prof. José Camargo



**GOVERNO DO ESTADO
DE SÃO PAULO**

Curso: Tecnologia em Sistemas para Internet

Período: 4º Semestre

Disciplina: Desenvolvimento para Servidores I

Professora: Profa. Dra. Ligia Rodrigues Prete

E-mail: ligia.prete@fatec.sp.gov.br

10 – Funções em PHP

Pasta da Aula

- Crie a pasta com o nome **“aula10”** dentro de **“Arquivos de código-fonte”**. Nela, crie os arquivos como **“Arquivo HTML”** e **“Página Web do PHP”**:

- 1) funcao_sem_par.php
- 2) funcao_com_par.php
- 3) funcao_com_retorno.php
- 4) funcao_sem_retorno.php
- 5) funcao_multiplos.php
- 6) funcao_valor.php
- 7) funcao_ref.php
- 8) funcoes.php

O que é uma função?

- Uma função é um pedaço de código com um objetivo específico, encapsulado sob uma estrutura única que recebe um conjunto de parâmetros e retorna um dado.
- Uma função é declarada uma única vez, mas pode ser utilizada diversas vezes. É uma das estruturas básicas para prover **reusabilidade**.
- **echo, gettype e var_dump** são exemplos de funções intrínsecas do PHP.

Sintaxe de função

`<?php`

```
function nome_funcao (arg1, arg2, ..., argN) {  
    instruções;  
    .  
    .  
    .  
    instruções;  
    return $valor_retornado;  
}
```

`?>`

Função **sem** parâmetros ou argumentos

- A função **bemVindo()** do próximo exemplo não possui parâmetros (ou argumentos) em sua declaração, portanto é uma função sem parâmetros (ou argumentos).

Conteúdo funcao_**sem**_par.php

```
12 <body>
13     <?php
14
15     //declarando a função
16     function bemVindo() {
17         echo "<p align='center'> Olá, seja bem vindo! </p>";
18     }
19
20     echo "<h3>Invocando a função bemVindo() </h3>";
21     bemVindo();
22
23     echo "<h3>Função bemVindo() dentro do laço For </h3>";
24     //usando a função dentro de uma estrutura for:
25     for ($i = 1; $i <= 5; $i++) {
26         bemVindo();
27     }
28     ?>
29 </body>
```

Invocando a função bemVindo()

Olá, seja bem vindo!

Função bemVindo() dentro do laço For

Olá, seja bem vindo!

Olá, seja bem vindo!

Olá, seja bem vindo!

Olá, seja bem vindo!

Olá, seja bem vindo!

Função **com** parâmetros ou argumentos

- São valores e/ou variáveis que deverão ser passados ou enviados à função para que esta os utilize para sua execução.
- Para que uma função possua parâmetros (ou argumentos), é necessário, em sua declaração, informar quais parâmetros (ou argumentos) ela necessita para ser executada corretamente.

Conteúdo funcao_com_par.php

```
<html>
  <head>
    <meta charset="UTF-8">
    <title></title>
  </head>
  <body>
    <?php
      3  4
      function soma($a, $b) {
        $s = $a + $b;
        echo "<p>A soma = $s</p>";
      }
      soma(3, 4);
      soma(8, 2);
      $x = 9;
      $y = 15;
      soma($x, $y);
    ?>
  </body>
</html>
```

A soma = 7

A soma = 10

A soma = 24

Funções em PHP

- Funções **com** e **sem** retorno

Nos exemplos anteriores, as funções simplesmente executam uma tarefa, ou seja, não há um retorno de informação de forma direta.

É possível criar funções que simplesmente retornam valores, assim pode-se “pegar” esses valores e utilizá-los após a invocação da função.

Conteúdo: funções **com** e **sem** retorno

funcao_**com**_retorno.php

```
7 <html>
8   <head>
9     <meta charset="UTF-8">
10    <title></title>
11  </head>
12  <body>
13    <?php
14
15    function soma($a, $b) {
16      $s = $a + $b;
17      return $s;
18    }
19
20    echo "<h3>Função com retorno</h3>";
21    $x = 20;
22    $y = 30;
23    $r = soma($x, $y);
24    echo "A soma entre $x e $y = $r";
25    ?>
26  </body>
27 </html>
```

Função com retorno

A soma entre 20 e 30 = 50

funcao_**sem**_retorno.php

```
7 <html>
8   <head>
9     <meta charset="UTF-8">
10    <title></title>
11  </head>
12  <body>
13    <?php
14
15    function soma($a, $b) {
16      $s = $a + $b;
17      echo "<p>A soma entre $a e $b = $s</p>";
18    }
19
20    echo "<h3>Função sem retorno</h3>";
21    $x = 20;
22    $y = 30;
23    soma($x, $y);
24    ?>
25  </body>
26 </html>
```

Função sem retorno

A soma entre 20 e 30 = 50

Função com **múltiplos** parâmetros ou argumentos

- Nos exemplos anteriores usamos 2 parâmetros (ou argumentos), mas podemos ter funções com parâmetros (ou argumentos) variáveis, ou seja, usar vários parâmetros (ou argumentos) em uma mesma função.

Conteúdo funcao_multiplos.php

```
12 <body>
13 <?php
14 function soma() {
15     /* func_get_args(): essa função pega todos os valores e coloca
16     * dentro de um vetor chamado $v. Assim, quando essa linha for
17     * executada irá ser criado um vetor com a quantidade de posições
18     * informadas. As posições sempre iniciam com zero.
19     * Por exemplo, $v[0] $v[1] $v[2] ... $v[n]
20     */
21     $v = func_get_args();
22     /* func_num_args(): essa função retorna o número de argumentos
23     * que foram passados.
24     */
25     $t = func_num_args();
26     $s = 0;
27     for ($i = 0; $i < $t; $i++) {
28         $s = $s + $v[$i];
29     }
30     return $s;
31 }
32
33 echo "<h3>Soma da função com múltiplos parâmetros</h3>";
34 $r = soma(3, 5, 2, 8, 9, 4);
35 echo "A soma = $r";
36 ?>
37 </body>
```

Soma da função com múltiplos parâmetros

A soma = 31

\$v[0]	\$v[1]	\$v[2]	\$v[3]	\$v[4]	\$v[5]	\$s = 31
3	5	2	8	9	4	

Funções que retornam parâmetros por **valor** e **referência**

- **Passagem por valor:** o valor da variável é colocado dentro do parâmetro. Qualquer mudança no parâmetro **não altera** o valor da variável.
- **Passagem por referência:** a variável é colocada dentro do parâmetro. Qualquer mudança no parâmetro **altera** o valor da variável.

Conteúdo funcao_valor.php

```
7 <html>
8   <head>
9     <meta charset="UTF-8">
10    <title></title>
11  </head>
12  <body>
13    <?php
14      function valor ($x){
15        $x = $x + 2;
16        echo "<p>O valor de x = $x</p>";
17      }
18
19      echo "<h3>Função que retorna parâmetro por valor</h3>";
20      $a = 3;
21      valor($a);
22      echo "<p>O valor de a = $a</p>";
23    ?>
24  </body>
25 </html>
```

Função que retorna parâmetro por valor

O valor de x = 5

O valor de a = 3

Conteúdo funcao_ref.php

```
7 <html>
8 <head>
9     <meta charset="UTF-8">
10    <title></title>
11 </head>
12 <body>
13     <?php
14
15         function referencia(&$x) {
16             $x = $x + 2;
17             echo "<p>O valor de x = $x</p>";
18         }
19
20         echo "<h3>Função que retorna parâmetro por referência</h3>";
21         $a = 3;
22         referencia($a);
23         echo "<p>O valor de a = $a</p>";
24     ?>
25 </body>
26 </html>
```

\$a

Função que retorna parâmetro por referência

O valor de x = 5

O valor de a = 5

Arquivos Externos

- Em qualquer linguagem de programação é comum a requisição (ou inclusão) de arquivos externos, ou seja, utilizar funcionalidades externas ao script que está sendo desenvolvido.
- É possível por exemplo, criar um arquivo contendo diversas funções e utilizar tais funções sem a necessidade de redeclará-las no script corrente, pois basta fazer uma requisição (ou inclusão) ao arquivo que contém as funções.
- No PHP existem 4 comandos que fazem a inclusão de arquivos externos:
 - ✓ `include(arquivo)`
 - ✓ `require(arquivo)`
 - ✓ `include_once(arquivo)`
 - ✓ `require_once(arquivo)`

Requisição de Arquivos

→ include (arquivo)

A instrução include() inclui e avalia o arquivo informado. Seu código (variáveis, objetos e arrays) entra no escopo do programa, tornando-se disponível a partir da linha em que a inclusão ocorre. Se o arquivo não existir, produzirá uma mensagem de advertência (**warning**).

→ require (arquivo)

Idêntico ao include. Difere somente pela manipulação de erros. Enquanto o include produz uma warning, o require produz uma mensagem de **Fatal Error** caso o arquivo não exista.

Requisição de Arquivos

→ `include_once` (arquivo)

Funciona da mesma maneira que o comando `include`, a não ser que o arquivo informado já tenha sido incluído, não refazendo a operação (o arquivo é incluído apenas uma vez). Este comando é útil em casos em que o programa pode passar mais de uma vez pela mesma instrução. Assim evitará sobreposições, redeclarações, etc.

→ `require_once` (arquivo)

Funciona da mesma maneira que o comando `require`, a não ser que o arquivo informado já tenha sido incluído, não refazendo a operação (o arquivo é incluído apenas uma vez). Este comando é útil em casos em que o programa pode passar mais de uma vez pela mesma instrução. Assim poderá evitar sobreposições, redeclarações, etc.

Reusabilidade de Funções

- Podemos criar funções externas que não ficam dentro do arquivo PHP exclusivamente. Essas funções podem ser reutilizadas por vários arquivos PHP.

Função externa

```
<?php  
    function nome_funcao (arg1, arg2,..., argN){  
        instruções ... ;  
        return $valor_retornado;  
    }  
?>
```



Conteúdo funcoes.php

```
<?php

//Função sem parâmetros ou argumentos
function texto() {
    echo "<p>Esta função mostra o texto sem argumento</p>";
}

//Função com parâmetros ou argumentos
function mostraValor($valor) {
    echo "<p>Esta função mostra o valor informado com argumento = $valor</p>";
}

?>
```

Conteúdo funcao_mostra.php

```
<html>
  <head>
    <meta charset="UTF-8">
    <title></title>
  </head>
  <body>
    <?php
      include "funcoes.php";
      echo "<h1> Testando funções </h1>";
      texto();
      mostraValor(5);
    ?>
  </body>
</html>
```

Testando funções

Esta função mostra o texto sem argumento

Esta função mostra o valor informado com argumento = 5